AP-HRRN: An Aging, Preemptive Variant of HRRN Scheduling

Operating Systems course project · Group 11

Team members:

Name	Registration number
Aryanish Singh Rathore	AP24110011056
Bhumi Sengar	AP24110010346
Surya Kanta Patra	AP24110011071
Subhanshu Singh	AP24110011104
Jainesh Bharti	AP24110010515

1. Why we did this

CPU scheduling is really just the question of who gets the processor next, and the answer decides how fast and how fair a computer feels. For this project we took two well-known algorithms, Shortest Job First (SJF) and Highest Response Ratio Next (HRRN), worked out where each one struggles, and built our own algorithm called AP-HRRN (Aging Preemptive HRRN) to patch those gaps.

Everything in this report comes from a Python simulation. The code, the graphs and the CSV files are all produced by the same scripts, and the random workloads use fixed seeds, so if you re-run the project you'll get exactly the same numbers.

2. What's wrong with SJF and HRRN

SJF always picks the job with the shortest burst. That keeps the average waiting time really low, but there's a catch: if short jobs keep showing up, a long job can get pushed back again and again. That's starvation. SJF also needs to know burst times in advance, and a real OS never knows them exactly.

HRRN tackles starvation with a response ratio, (waiting time + burst) / burst. The longer a job waits, the higher its ratio climbs, so it eventually gets its turn. The problem is that HRRN is non-preemptive. Once a long job starts running, nothing can interrupt it, so short jobs that arrive right after it are stuck waiting. In our tests HRRN also ended up with a clearly higher average waiting time than SJF.

So the gap we wanted to fill was this: keep HRRN's fairness, get closer to SJF's quick response, and stop depending on exact burst times.

3. Our idea: AP-HRRN

AP-HRRN is preemptive and makes a fresh scheduling decision every time unit. It works through four ideas put together.

1. Response ratio on the remaining time. Instead of the full burst, the score looks at how much time is estimated to be left:

score = (waited + est_remaining) / est_remaining + boost × floor(waited / L)

Here `waited` is how long a process has been ready but not running, `est_remaining` is the estimated burst minus what has already run (never below 1), `L` is the aging wait limit, and `boost` is the bonus added for every full `L` units of waiting. A short remaining time pushes the score up, like SJF. A long wait pushes it up too, like HRRN.

2. Careful preemption. A waiting process can take the CPU from the running one only if two things are true: its score is at least `k` times the running process's score, and its estimated remaining time is smaller. The factor `k` stops the CPU from flipping back and forth over tiny differences.

3. No endless bumping. Once the running process has itself waited `L` units or more, it can't be preempted anymore. This is our starvation guard, so nobody gets pushed around forever.

4. Estimating the burst. The simulator can run in two modes. In `exact` mode it uses the true burst, which is the ideal case. In `exp` mode it predicts the burst with exponential averaging, tau_new = alpha × last_burst + (1 − alpha) × tau_old, starting from tau0 = 6. That's much closer to what a real system could actually do.

The defaults for all the main experiments are alpha = 0.5, tau0 = 6, k = 1.5, L = 24 and boost = 10, with the exact estimator.

A quick example

Take five processes: P1 (arrival 0, burst 8), P2 (1, 7), P3 (2, 2), P4 (9, 1) and P5 (10, 1).

Algorithm	Avg waiting	Avg response	Max waiting	Context switches
SJF	3.80	3.80	11	4
HRRN	6.20	6.20	9	4
AP-HRRN	4.40	2.20	11	5

Here AP-HRRN lets P3 jump in early, so the response time drops to 2.20, the best of the three. The price is one extra context switch, and its average waiting time lands between SJF and HRRN.

4. How we tested it

The simulator is written in Python. `scheduler.py` has the three algorithms, `process.py` has the workloads and the metric calculations, `compare.py` makes the comparison tables and graphs, and `tuning.py` runs the parameter sweeps. There are 18 automated tests, and all of them pass.

We used two kinds of workloads:

- **Three fixed scenarios:** a normal mixed load, a starvation-prone load (one long job with burst 20 and a steady stream of short jobs arriving), and a load with lots of short jobs.
- **Random workloads:** 40 randomly generated sets of 15 processes each (arrival between 0 and 40, burst between 1 and 15, seeds 100 to 139). We report the average over all 40.

For every run we measured average waiting time, average turnaround time, average response time, maximum waiting time, number of context switches and CPU utilization.

5. What we found

Scenario	Algorithm	Avg waiting	Avg turnaround	Avg response	Max waiting	Context switches
Mixed	SJF	11.90	18.10	11.90	47	9
Mixed	HRRN	13.60	19.80	13.60	47	9
Mixed	AP-HRRN	15.50	21.70	15.50	44	9

Scenario	Algorithm	Avg waiting	Avg turnaround	Avg response	Max waiting	Context switches
Starvation	SJF	10.50	13.32	10.50	41	21
Starvation	HRRN	26.86	29.68	26.86	39	21
Starvation	AP-HRRN	18.14	20.95	17.05	39	22
Many short	SJF	2.00	4.62	2.00	27	15
Many short	HRRN	8.19	10.81	8.19	16	15
Many short	AP-HRRN	5.56	8.19	4.62	18	17
Random (avg of 40)	SJF	26.31	34.34	26.31	85.08	14.00
Random (avg of 40)	HRRN	28.83	36.85	28.83	78.20	14.00
Random (avg of 40)	AP-HRRN	31.22	39.25	30.70	75.90	14.78

CPU utilization is 100% in all the fixed scenarios and 99.86% on average for the random runs, so none of the algorithms wastes CPU time.

Charts for each scenario

Each chart shows all six metrics for SJF, HRRN and AP-HRRN side by side.

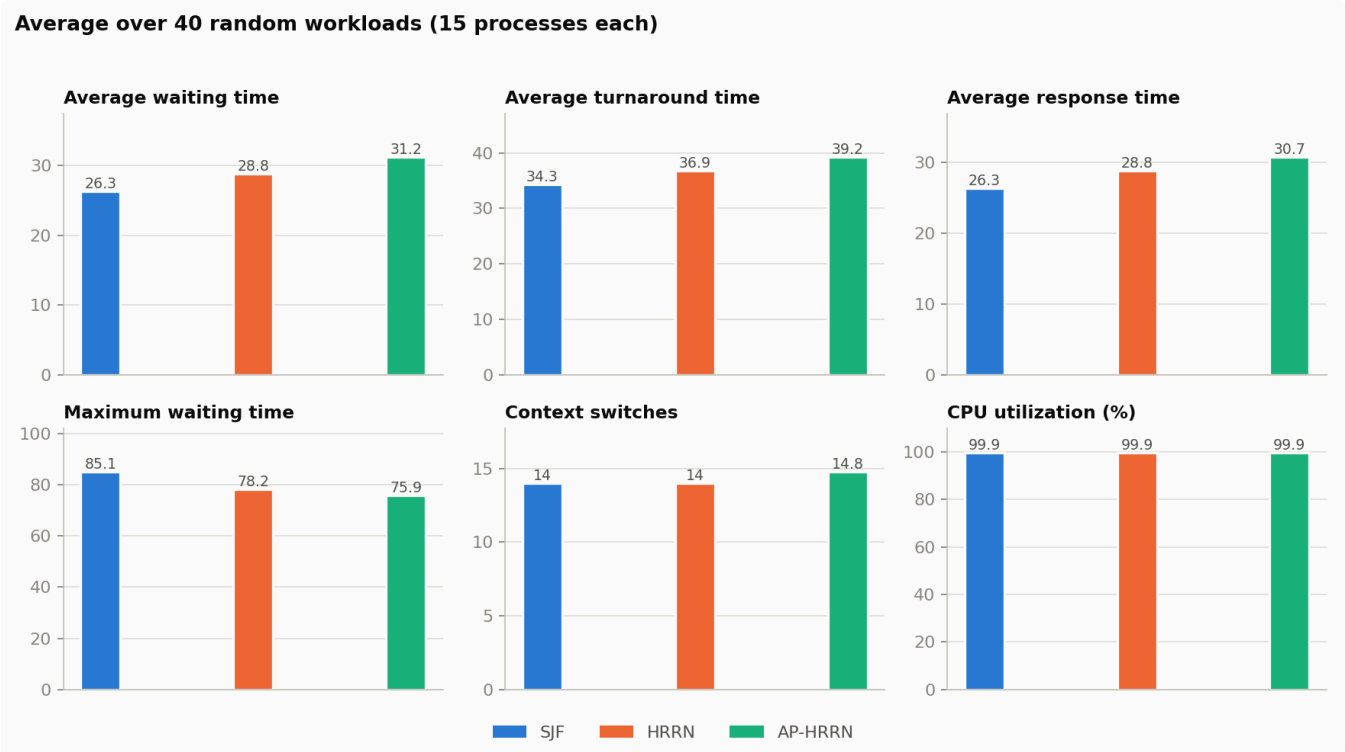


Figure 1. All six metrics, averaged over 40 random workloads.

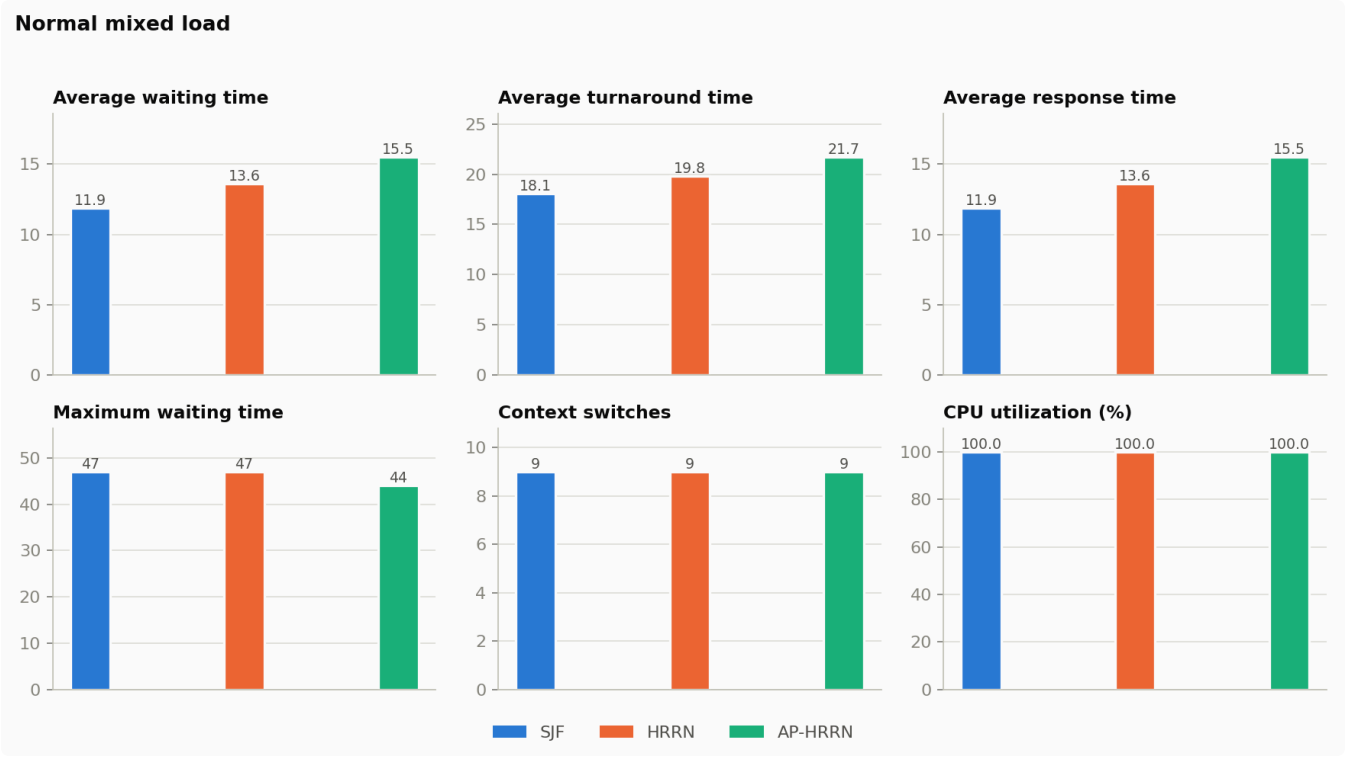


Figure 2. All six metrics on the normal mixed load.

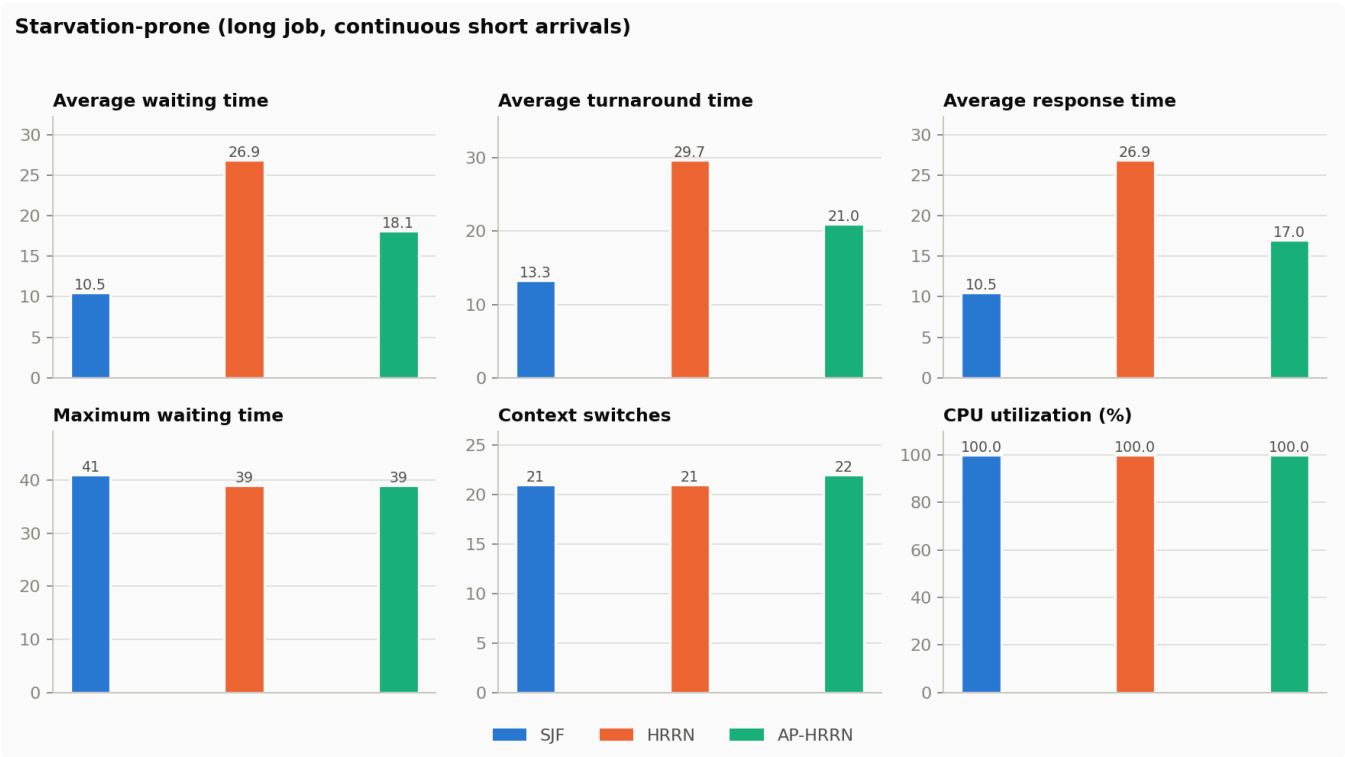


Figure 3. All six metrics on the starvation-prone load.

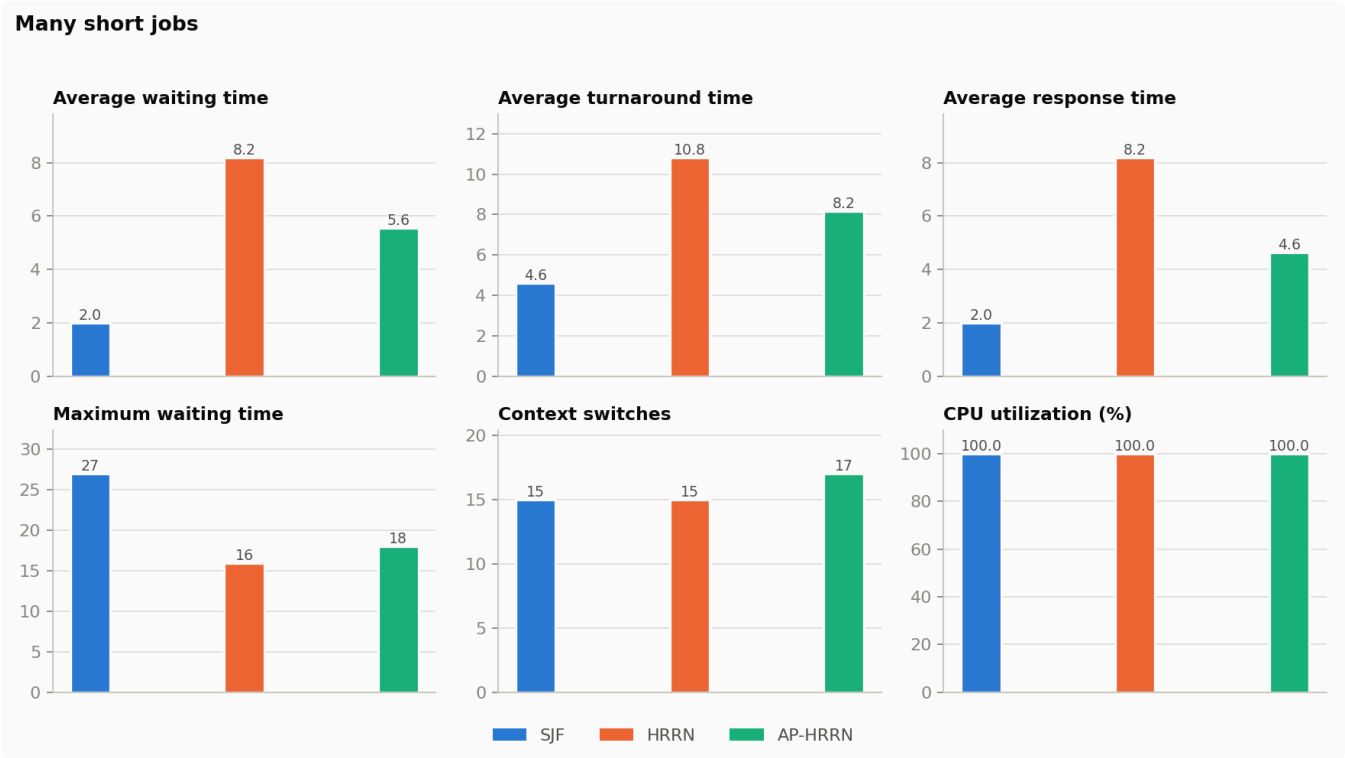


Figure 4. All six metrics on the many-short-jobs load.

So what do the numbers say?

Let's be honest about it: **AP-HRRN does not win on average waiting time.** SJF has the lowest average waiting in every scenario, which makes sense because SJF is optimal for that metric when it knows the exact bursts. AP-HRRN also trails HRRN in the mixed scenario (15.50 vs 13.60) and on the random average (31.22 vs 28.83).

Here's where AP-HRRN does help:

- **Max waiting time (the worst-off process).** On the random average it's the lowest of the three at 75.90, against 78.20 for HRRN and 85.08 for SJF. In the mixed scenario it's 44, while both of the others sit at 47.
- **Starvation scenario.** Under SJF the long job waits 41 units, because it only runs after all the short jobs are done. AP-HRRN and HRRN bring that down to 39. On top of that, AP-HRRN's average waiting (18.14) is far better than HRRN's (26.86), since it doesn't leave the short jobs stuck behind the long one.
- **Many short jobs.** AP-HRRN's average waiting (5.56) and response time (4.62) are clearly better than HRRN's (8.19 for both).

It does cost something: a few more context switches (for example 14.78 against 14 on the random average, and 17 against 15 on many short jobs), because preemption is allowed.

In short, AP-HRRN gives up a little average waiting time in exchange for fairness and a lower worst-case wait. That's a fair deal for a general-purpose system, but it isn't a free win, so we're not claiming one.

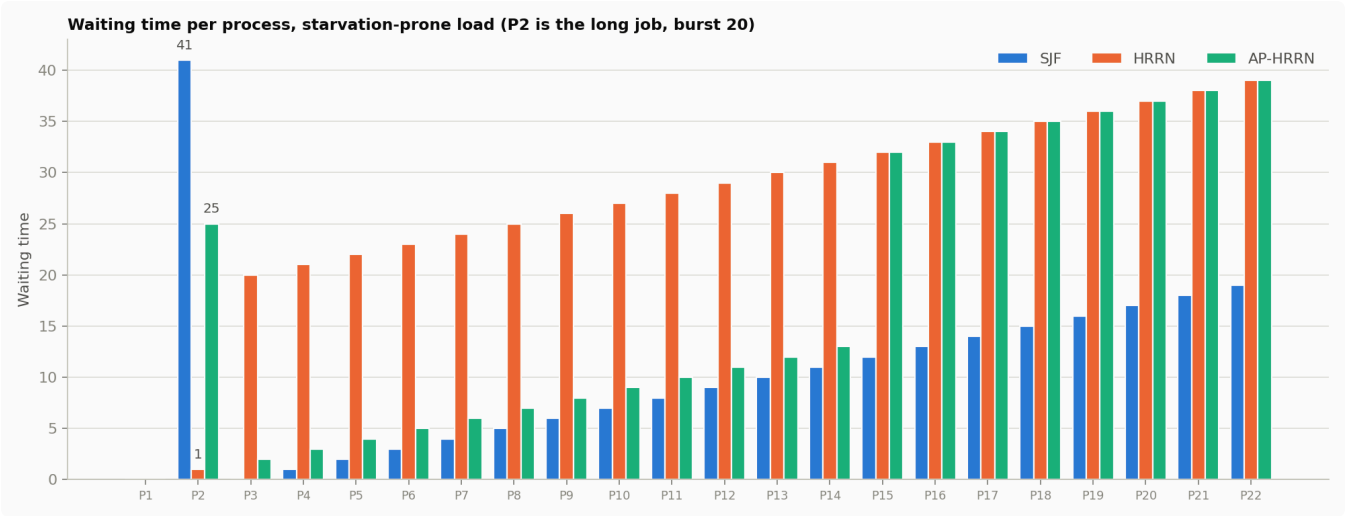


Figure 5. Waiting time of every process in the starvation-prone load.

Gantt charts

These show which process ran when under each algorithm. A process that appears in more than one block was preempted, which is easy to spot in AP-HRRN.

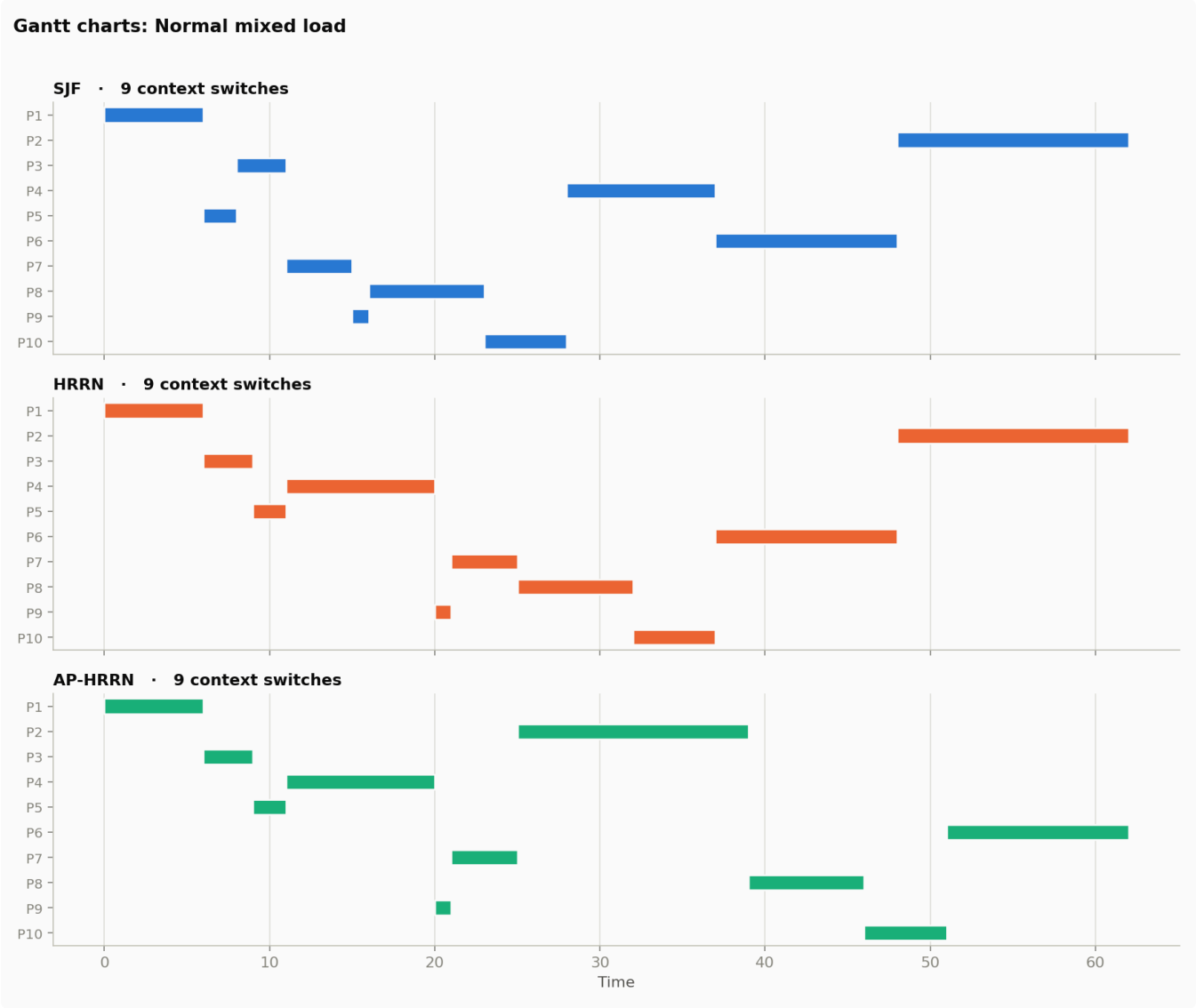


Figure 6. Gantt charts for the normal mixed load.

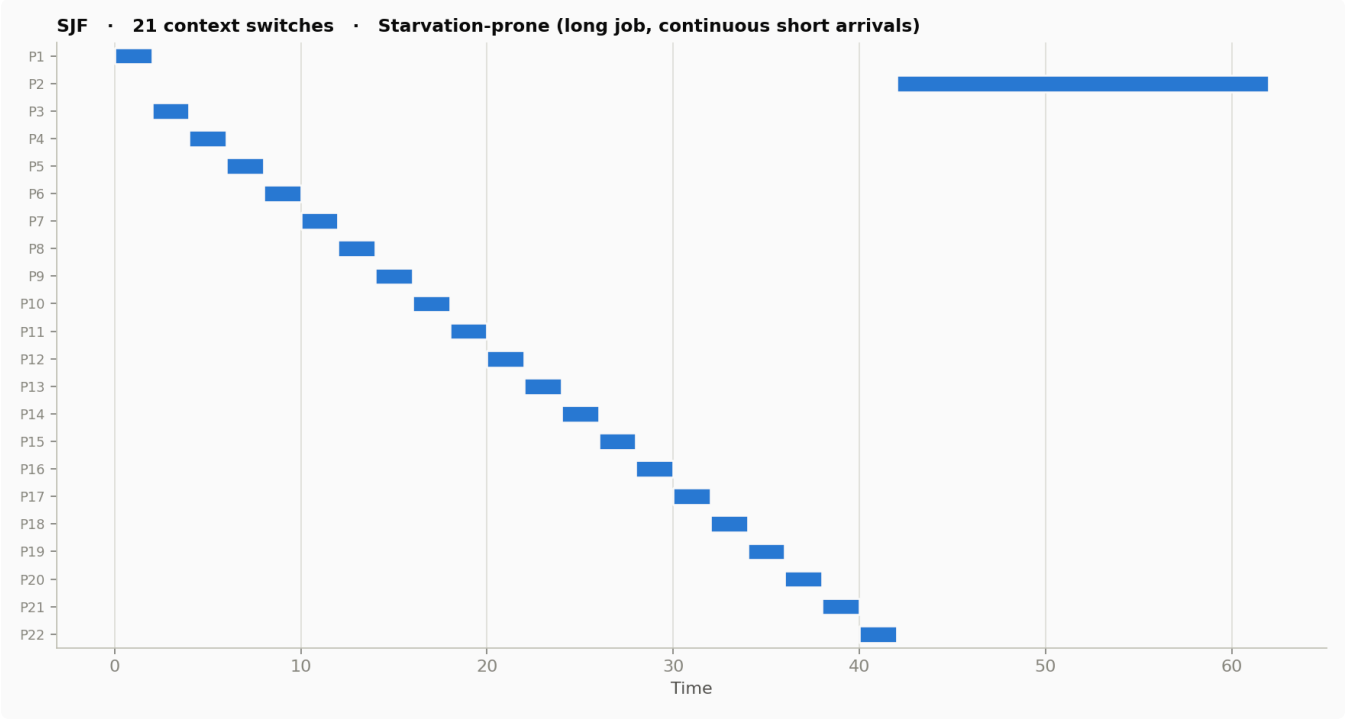


Figure 7a. Gantt chart for the starvation-prone load under SJF.

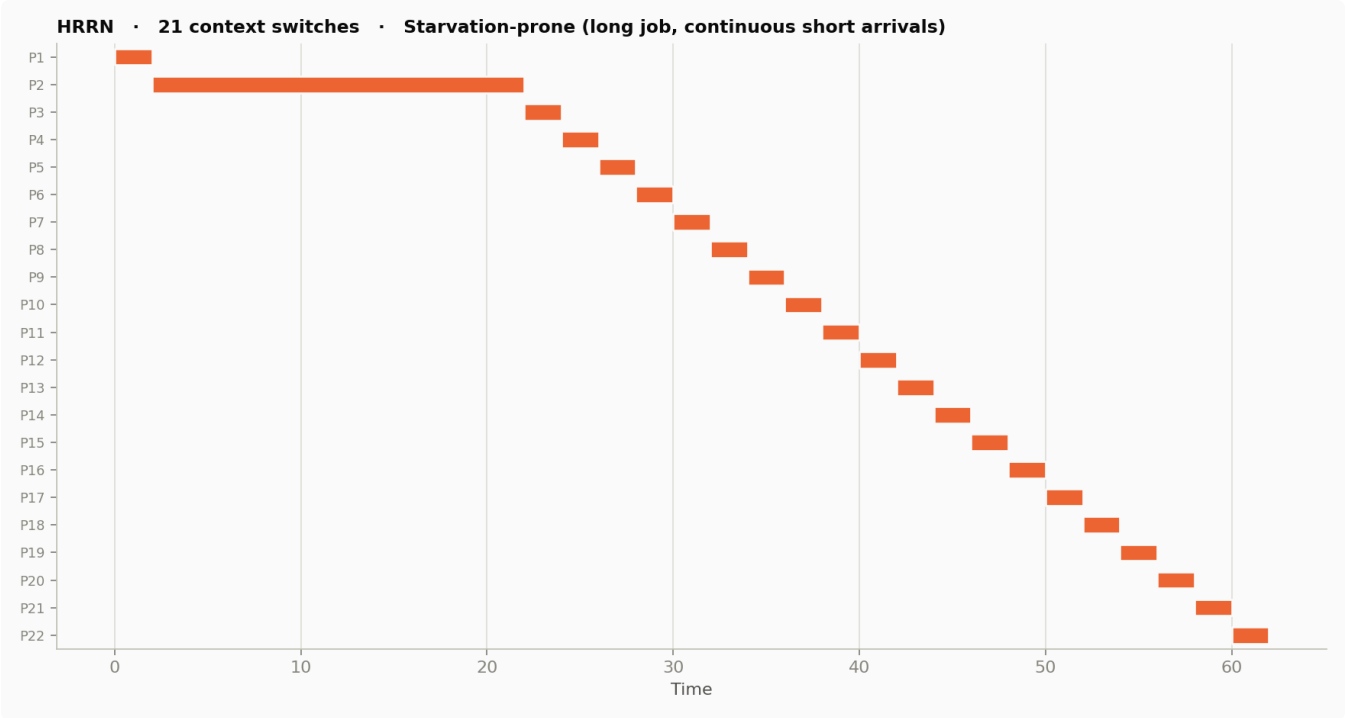


Figure 7b. Gantt chart for the starvation-prone load under HRRN.

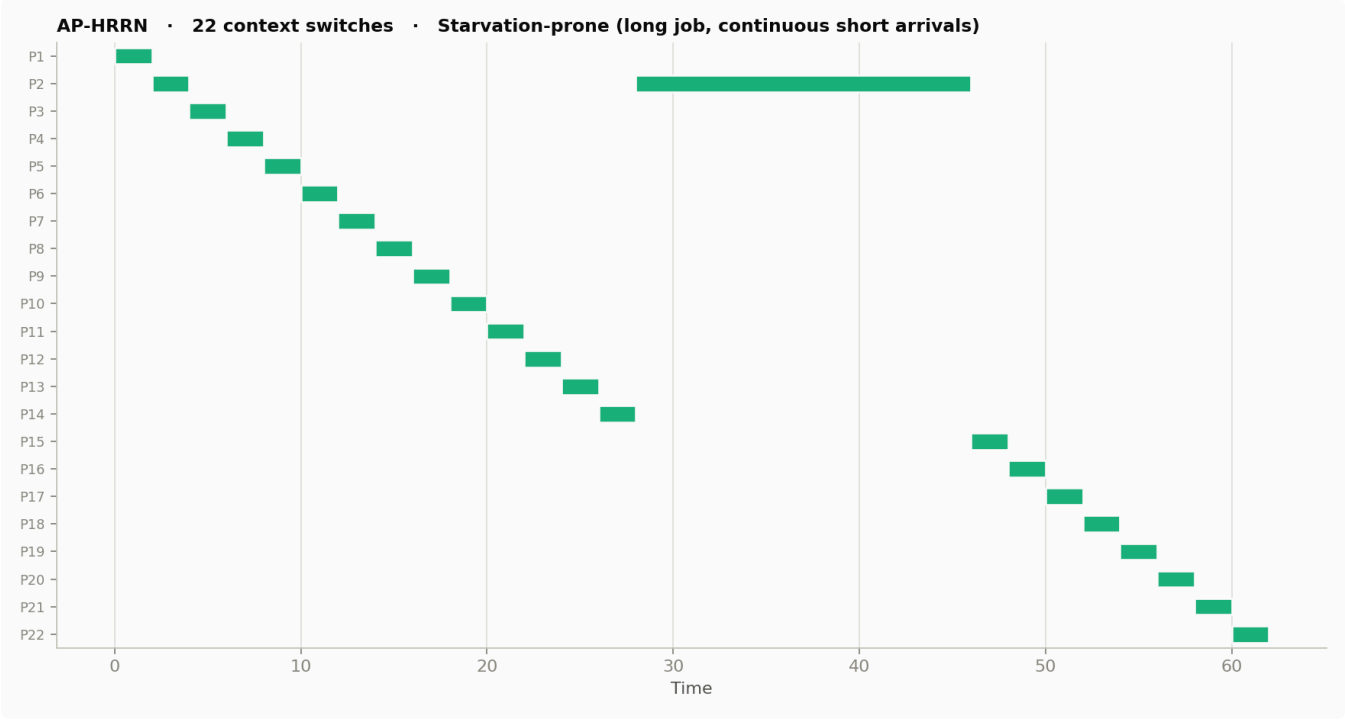


Figure 7c. Gantt chart for the starvation-prone load under AP-HRRN.

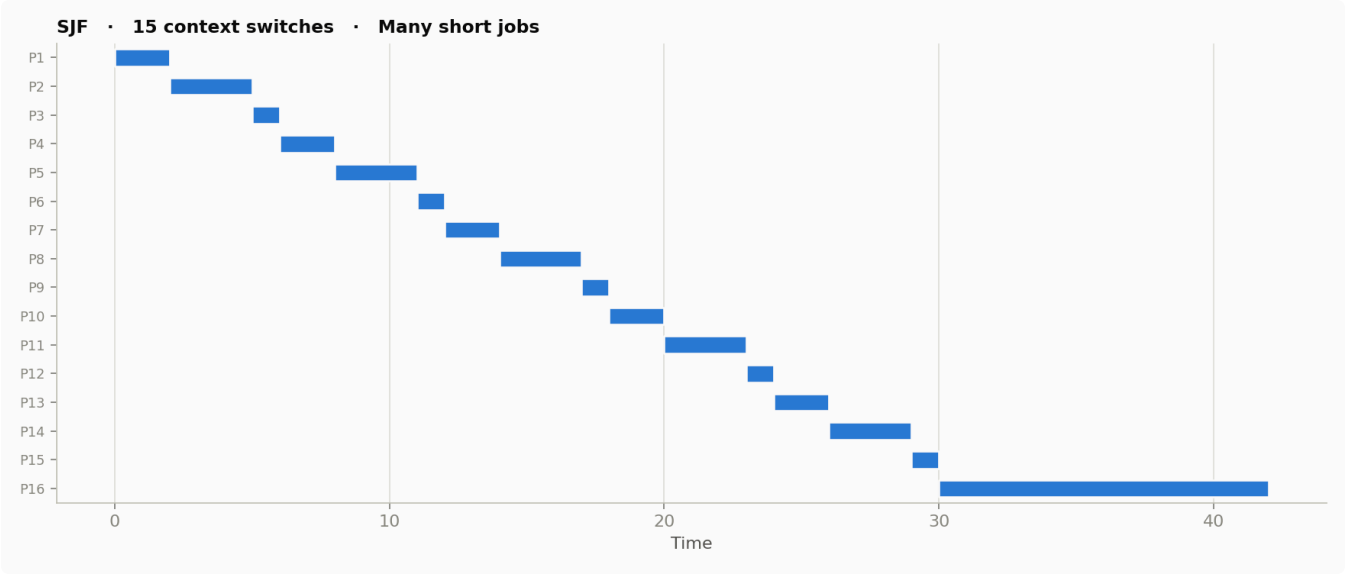


Figure 8a. Gantt chart for the many-short-jobs load under SJF.

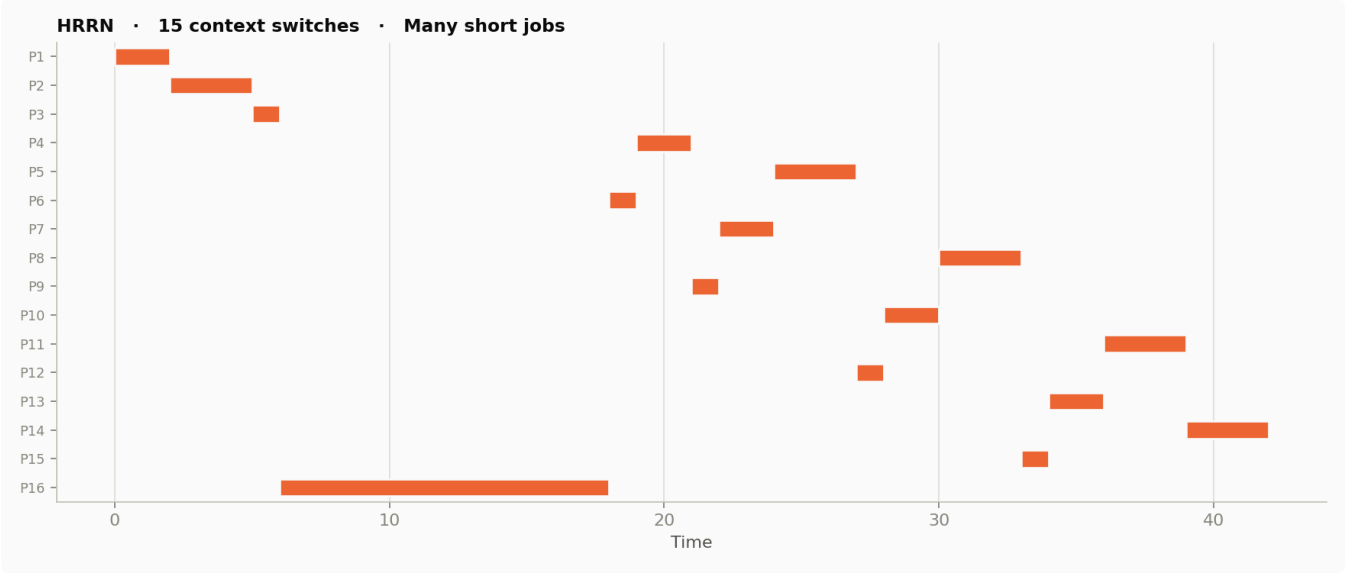


Figure 8b. Gantt chart for the many-short-jobs load under HRRN.

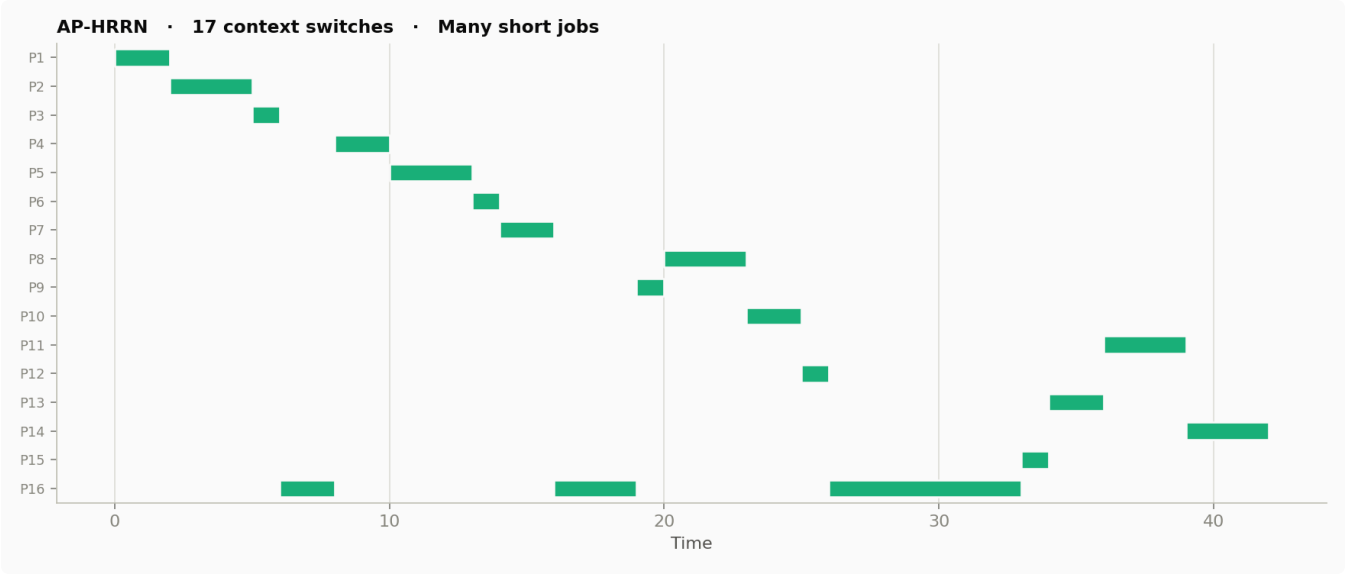


Figure 8c. Gantt chart for the many-short-jobs load under AP-HRRN.

6. Playing with the parameters

We changed one parameter at a time on the 40 random workloads and left the others at their defaults.

Preemption threshold k. A higher k means fewer preemptions. Context switches drop from 15.68 (k = 1.0) to 14.07 (k = 5.0), while average waiting only creeps up from 30.82 to 31.49. On the starvation scenario the effect is bigger, with average waiting going from 17.59 to 21.95. We stayed with k = 1.5 as a middle ground.

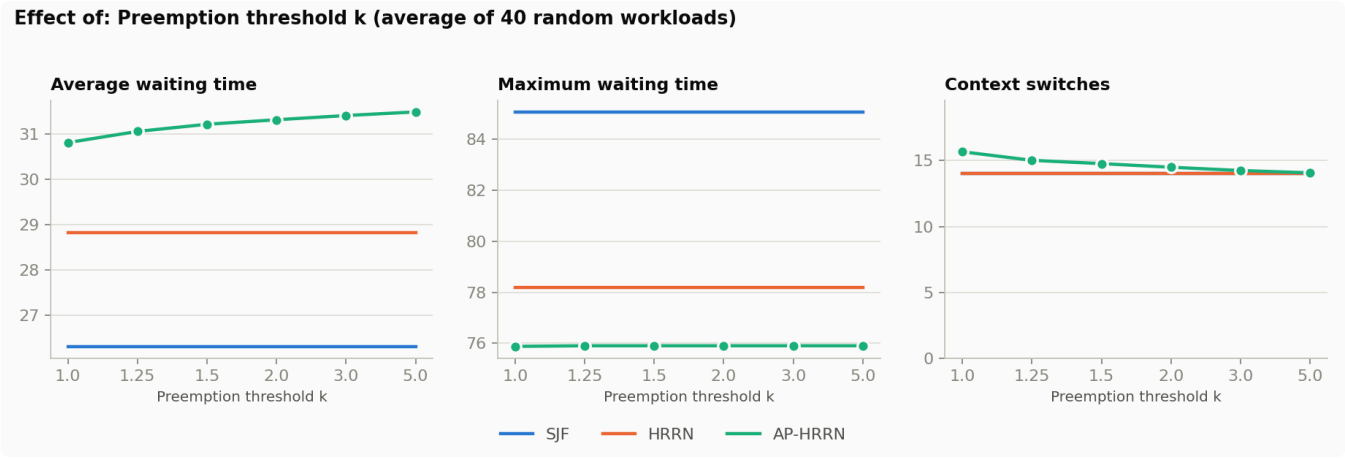


Figure 9. Effect of the preemption threshold k (average of 40 random workloads).

Aging wait limit L. A smaller L protects long jobs sooner, but it also switches off preemption sooner, so average waiting goes up (36.01 at L = 4, 31.22 at L = 24, and 28.62 with aging off). Max waiting is best around L = 12 (75.15) and gets worse, to 78.20, once aging is off. That's the same as HRRN, which shows the aging is what actually brings the worst-case wait down. In the starvation scenario every finite L gives a max waiting of 39, and turning aging off gives 41.

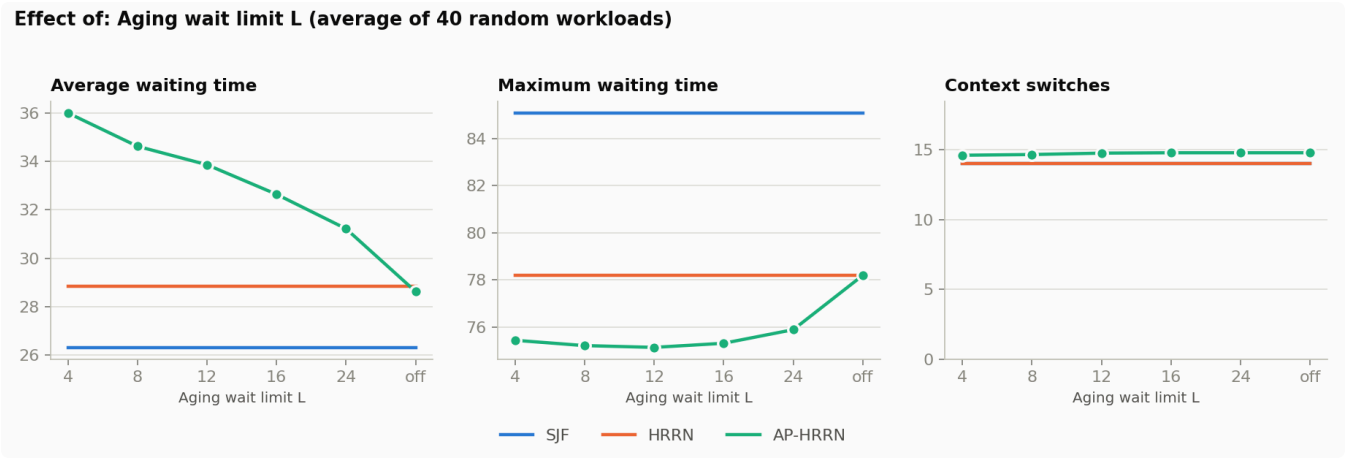


Figure 10. Effect of the aging wait limit L (average of 40 random workloads).

Aging boost. Boost 0 gives an average waiting of 28.62 and a max waiting of 78.20. From boost 10 onward the max waiting settles at 75.90 and average waiting at about 31.2 to 31.4, so pushing the boost higher doesn't buy anything more. Boost 10 is enough.

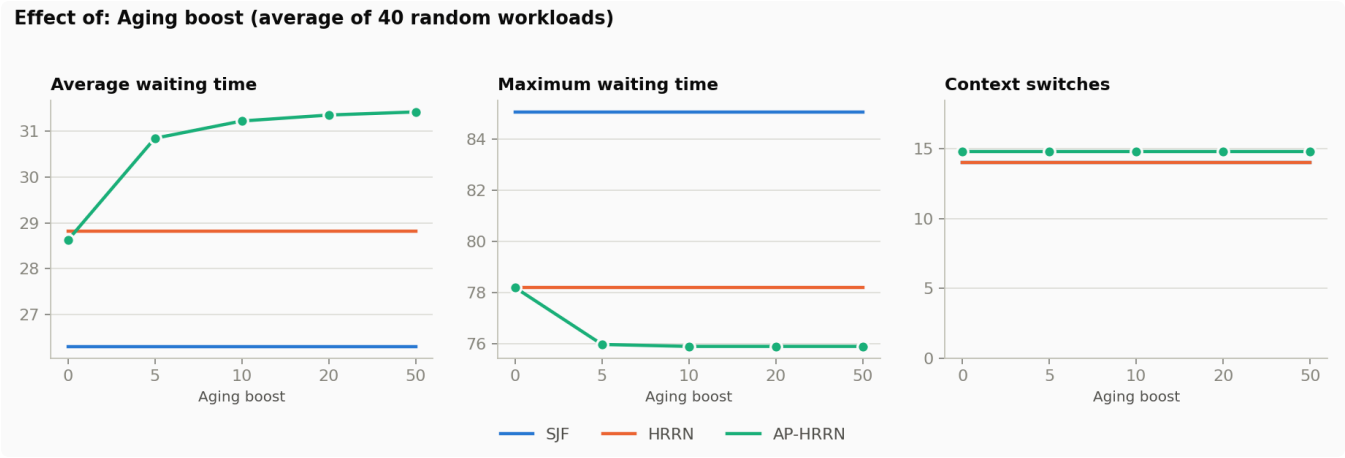


Figure 11. Effect of the aging boost (average of 40 random workloads).

Smoothing factor alpha (estimated bursts). With the exponential estimator, average waiting on the random workloads sits around 38.7 to 39.2 whatever alpha we pick, which is clearly worse than the 31.22 we get with exact bursts. Alpha itself hardly matters. The loss comes from the error in estimating bursts, not from AP-HRRN's logic.

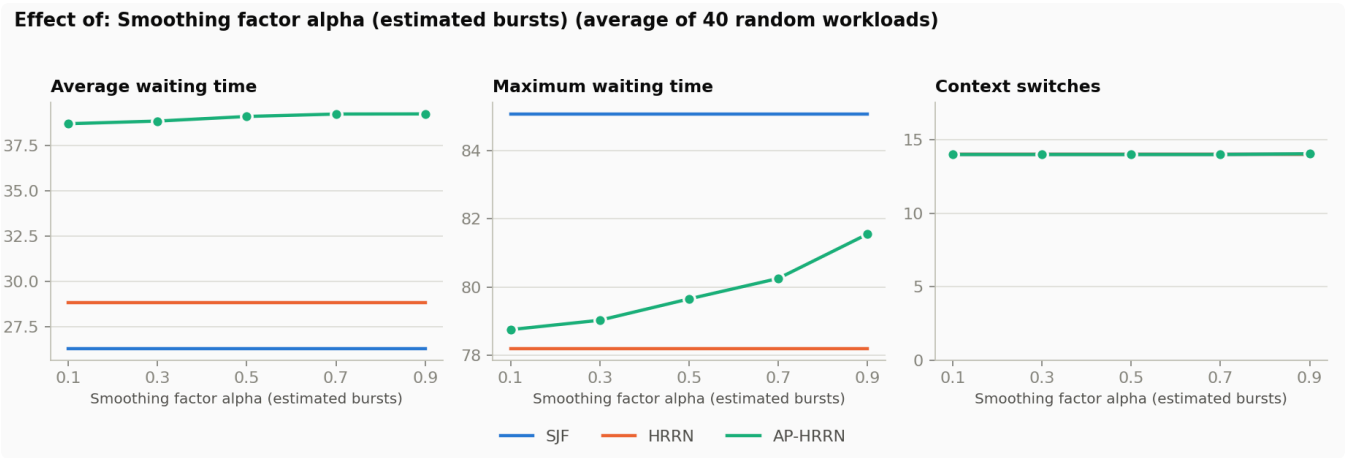


Figure 12. Effect of the smoothing factor alpha, using the exponential burst estimator.

7. Where it falls short

- The main comparison gives all three algorithms exact burst times, which is a fair ideal-case test. With estimated bursts (the realistic case) AP-HRRN loses ground, as the alpha sweep shows, and comparing it against SJF and HRRN running on exact bursts wouldn't be like for like.
- The simulation works in whole time units and doesn't charge any time for a context switch, so AP-HRRN's extra switches are counted but not paid for.
- We tested three hand-made scenarios and 40 random sets. More workload types, such as I/O-heavy processes, would give a fuller picture.

- We tuned the parameters on the same random set we report on, so the numbers may be a bit optimistic.

8. Wrapping up

AP-HRRN mixes HRRN's response-ratio idea with SJF-style preemption, an aging boost, and a rule that stops a long-waiting process from being preempted forever. In our simulations it gives the lowest maximum waiting time on random workloads, handles the starvation case much better than plain HRRN on average waiting, and responds faster than HRRN on many short jobs. In return, its average waiting is higher than SJF's, sometimes higher than HRRN's too, and it makes a few more context switches. So we see it as a fairness-focused scheduler, not a replacement that beats both of its parents on every metric.

Appendix: how to run it yourself

Run these from the project folder:

`pip install matplotlib pytest`

`python scheduler.py` for the demo, `python compare.py` for tables and graphs, `python tuning.py` for the parameter sweeps and `python -m pytest tests` for the tests. All graphs in `graphs/` and all CSVs in `results/` are regenerated by these scripts.